

Breaking Isolation

Last-level Cache Side-Channel Attacks on AWS Nitro Enclaves

Monder Rammouz, Quoc-Do Le

Technical University of Munich, Huawei Munich Research Center



TEEs protect from software threats ...

- Prime+Probe feasible on GCP across gVisor Containers¹



“Some instance sizes can share some LLC lines non-simultaneously.”²

Why AWS Nitro Enclaves?

AWS Nitro Enclaves enables customers to create isolated compute environments to further protect and securely process highly sensitive data such as personally identifiable information (PII), healthcare, financial, and intellectual property data within their Amazon EC2 instances. Nitro Enclaves uses the same Nitro Hypervisor technology that provides CPU and memory isolation for EC2 instances.

Can LLC be exploited to launch side-channel attacks against Enclaves in cloud?

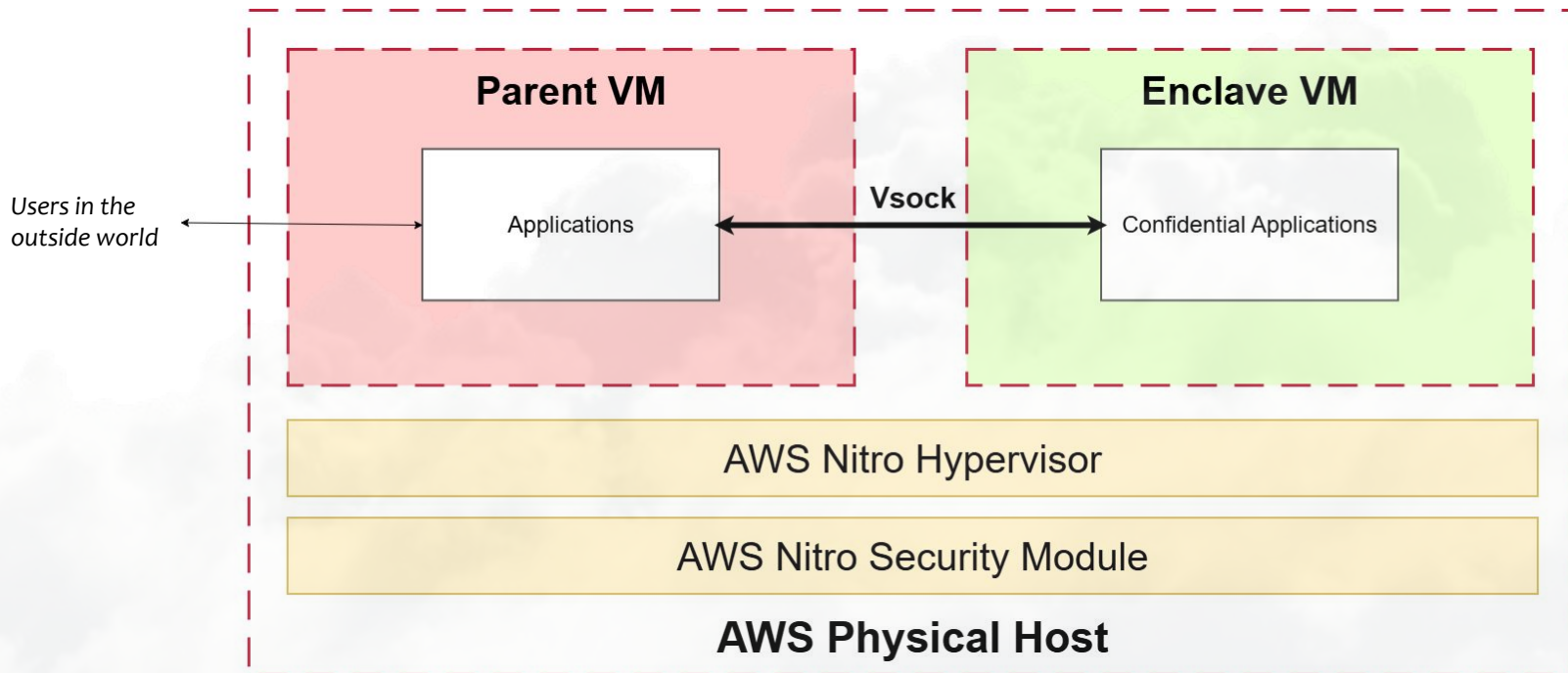
→ **PoC:** overcome AWS Nitro Enclave isolation & cloud challenges

¹ Zhao et al. 2022

² AWS Whitepaper – The Security Design of the AWS Nitro System (February 15, 2024)

Threat Model

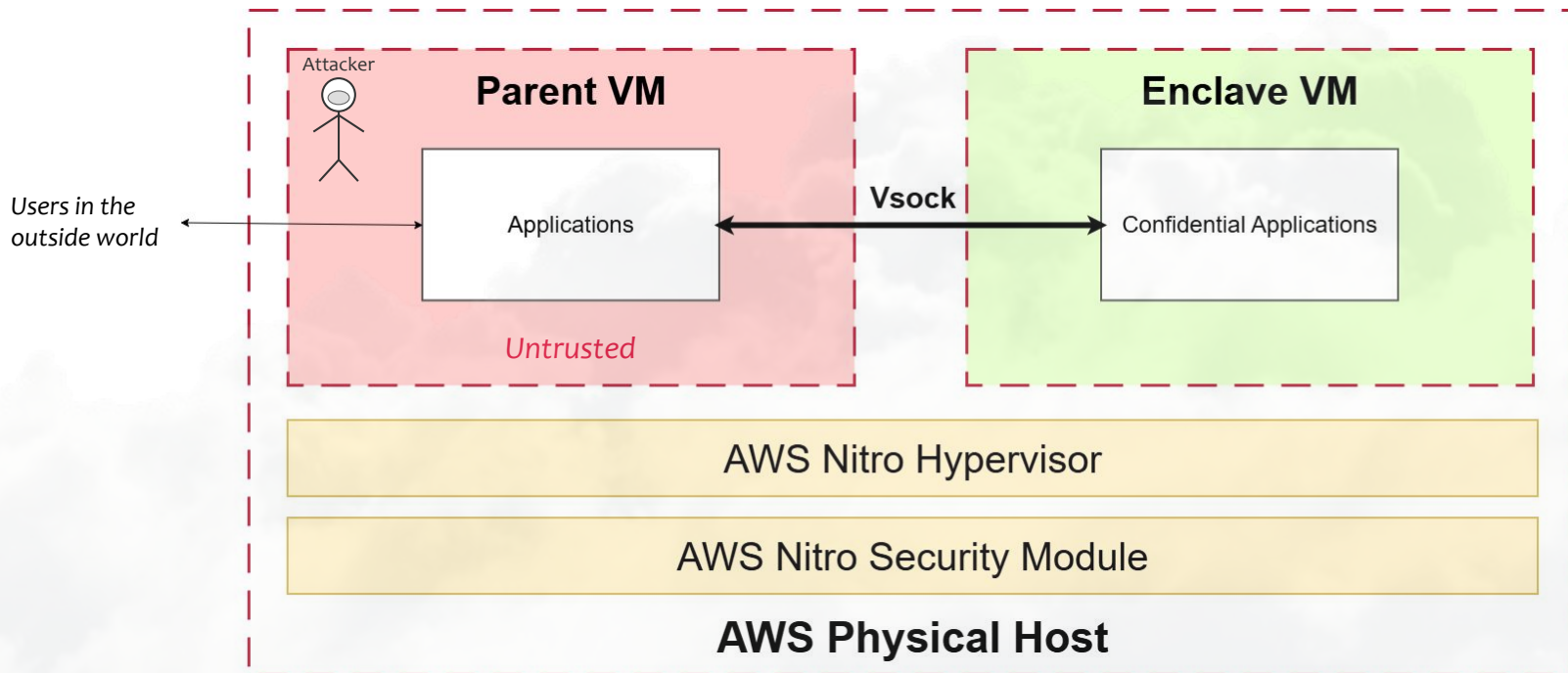
dedicated CPU cores (~~SMT sharing~~) + isolated memory region +
~~networking~~ → minimal attack surface



Threat Model

dedicated CPU cores (~~SMT sharing~~) + isolated memory region +
~~networking~~ → minimal attack surface

*Assumption 1: attacker
co-located in untrusted world*



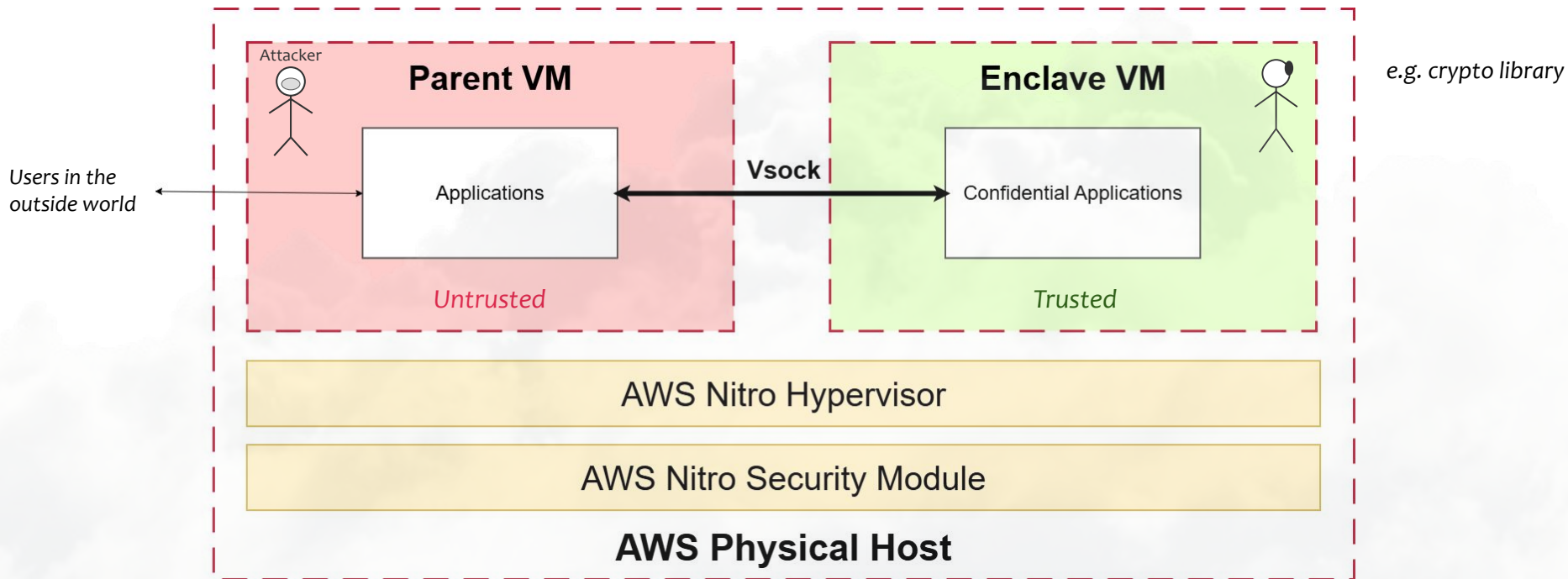
Threat Model

dedicated CPU cores (~~SMT sharing~~) + isolated memory region +
~~networking~~ → minimal attack surface

*Assumption 1: attacker
co-located in untrusted world*

*Assumption 2: victim is
running during the attack*

*Assumption 3: victim accesses
cache in a secret-dependent way*



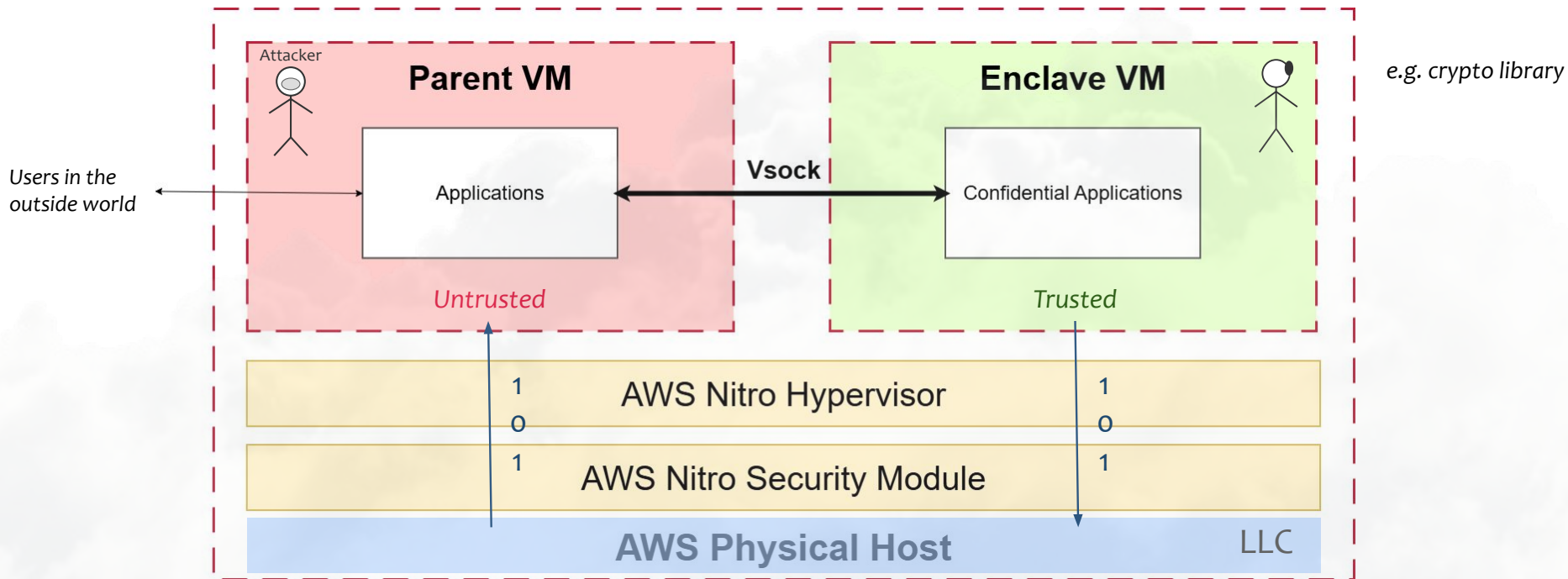
Threat Model

dedicated CPU cores (SMT sharing) + isolated memory region +
networking → minimal attack surface

*Assumption 1: attacker
co-located in untrusted world*

*Assumption 2: victim is
running during the attack*

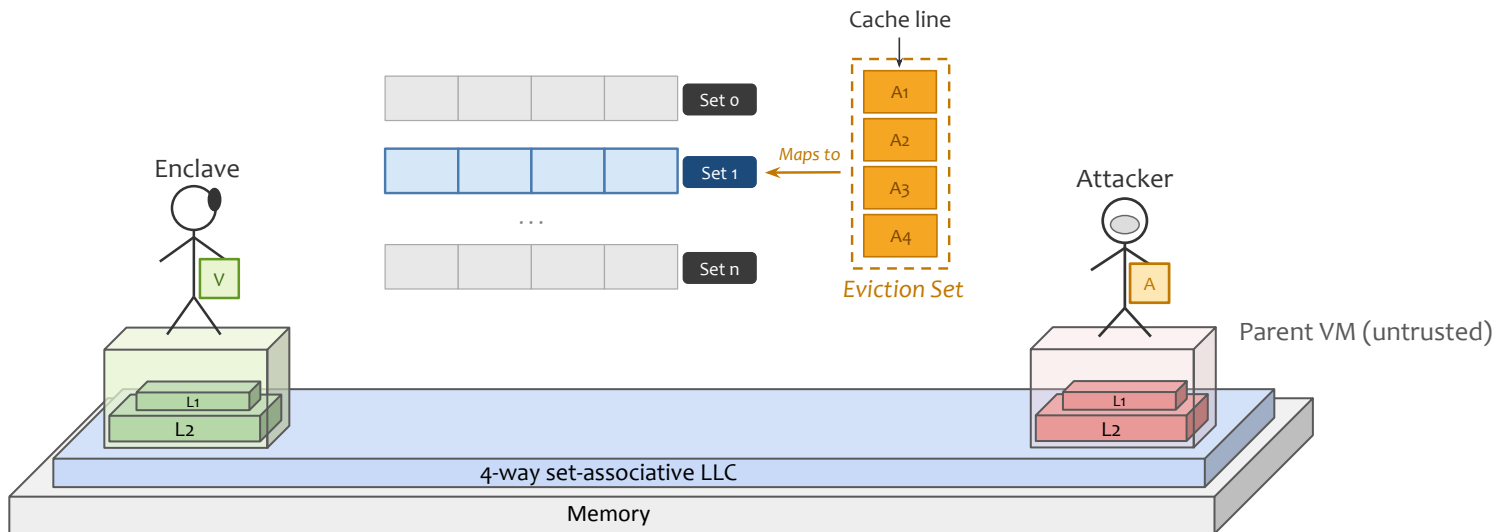
*Assumption 3: victim accesses
cache in a secret-dependent way*



LLC shared with parent VM and all co-tenants on NUMA node!

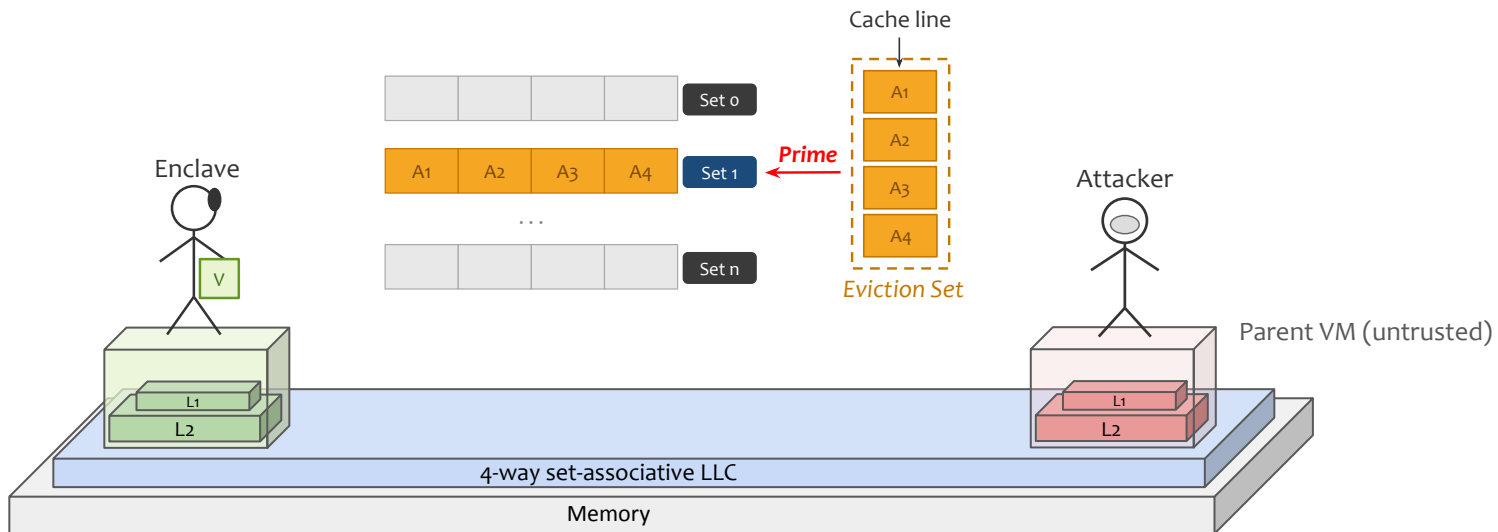
Prime+Probe

→ **eviction set** = smallest address group guaranteed to evict all lines of an LLC set



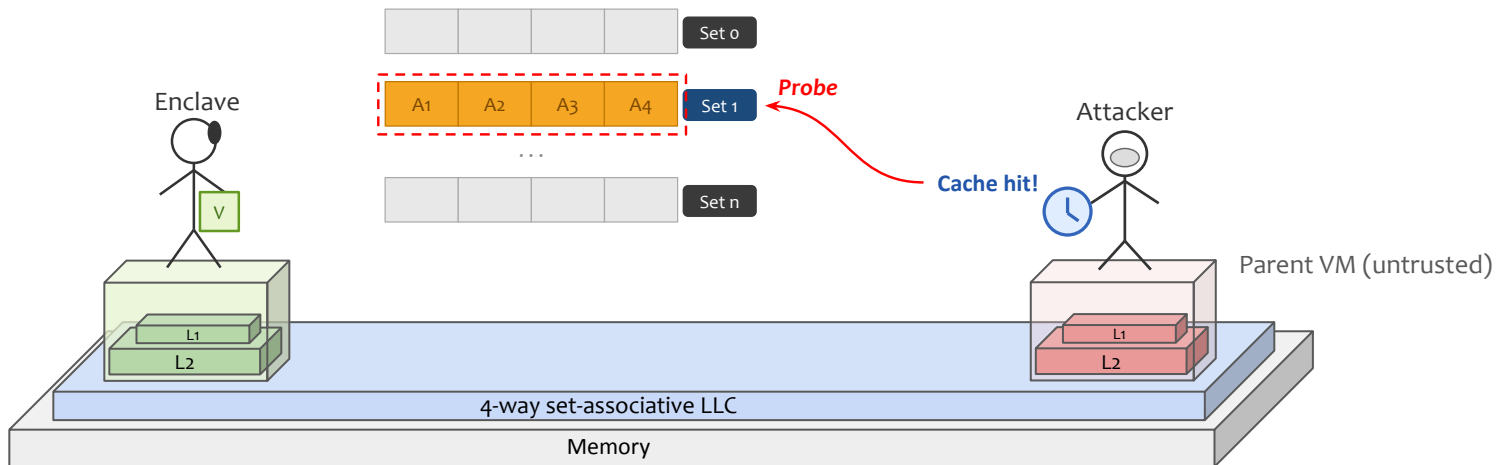
Prime+Probe

→ **eviction set** = smallest address group guaranteed to evict all lines of an LLC set



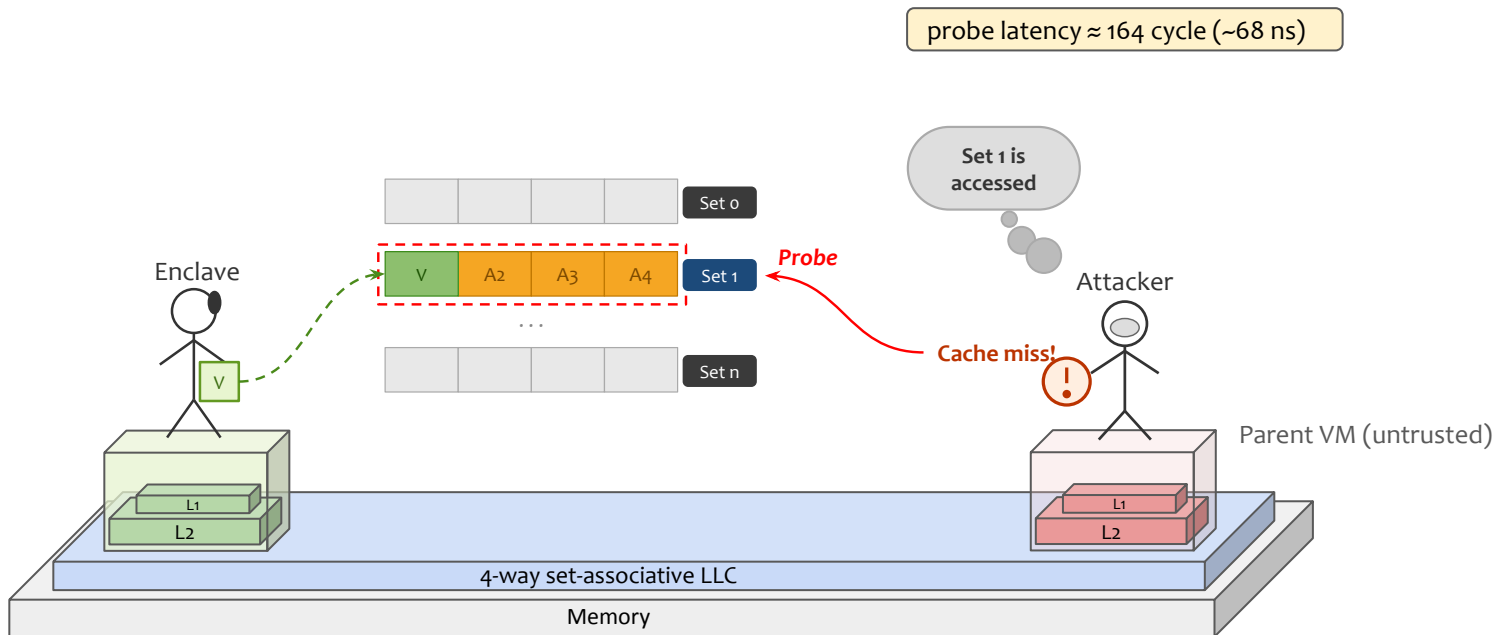
Prime+Probe

→ **eviction set** = smallest address group guaranteed to evict all lines of an LLC set



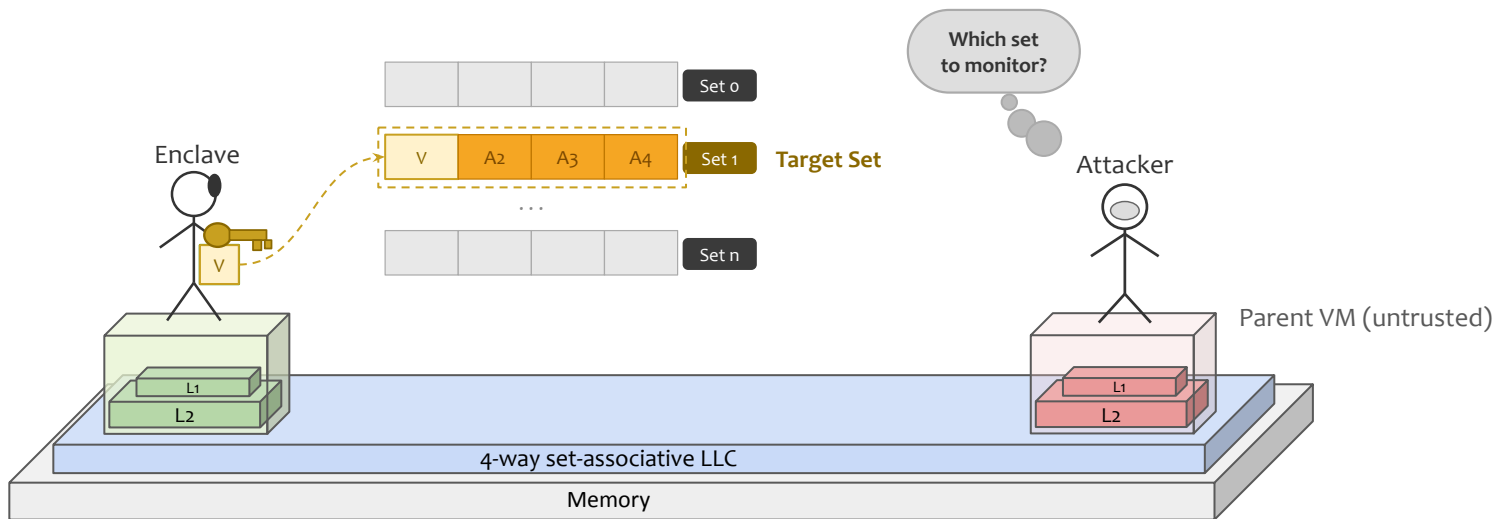
Prime+Probe

→ **eviction set** = smallest address group guaranteed to evict all lines of an LLC set



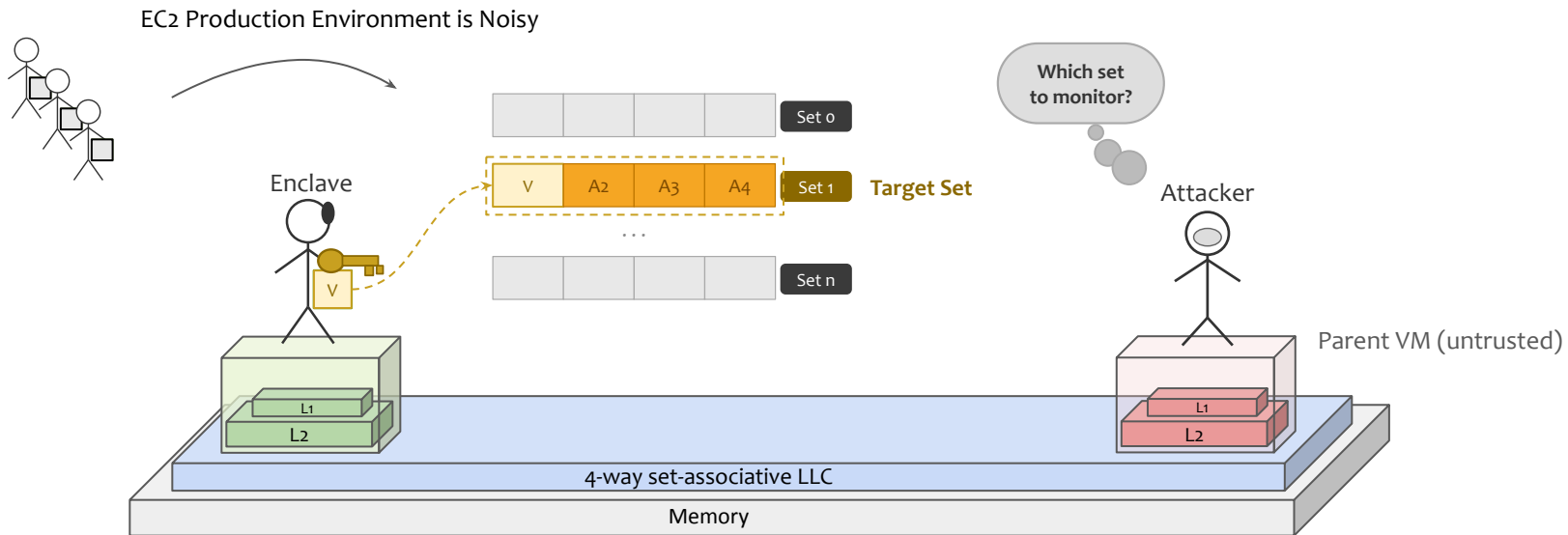
Prime+Probe

→ **eviction set** = smallest address group guaranteed to evict all lines of an LLC set

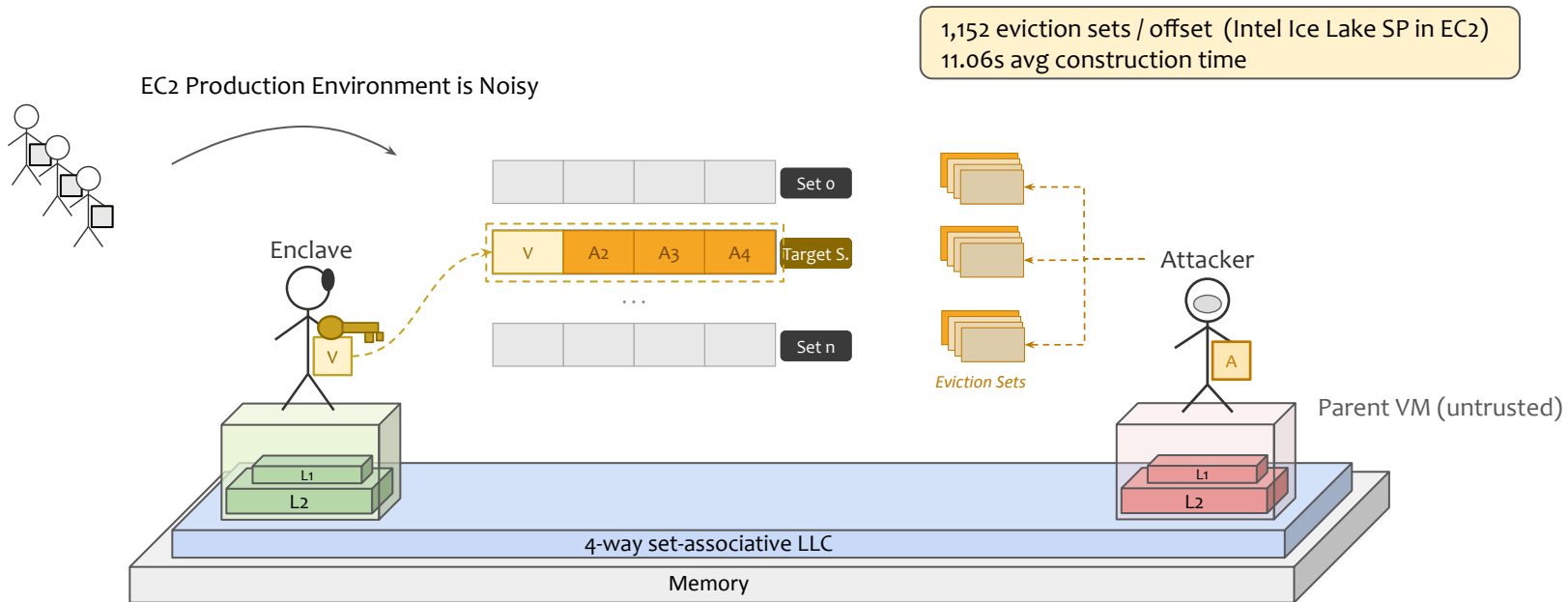


Prime+Probe

→ **eviction set** = smallest address group guaranteed to evict all lines of an LLC set



Building Eviction Sets



End-to-end Attack

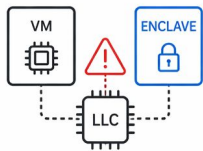
✓ Zhao et al., Last-Level Cache Side-Channel Attacks Are Feasible in the Modern Public Cloud

1. Building eviction sets
 2. Prime+Probe
 3. **Identifying target**
 4. **Key extraction**
- > New methods!

Our work: AWS Nitro Enclave information leakage in production



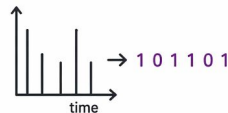
LLC-sharing Vulnerability



Enclave confidential app monitoring disclosed to AWS



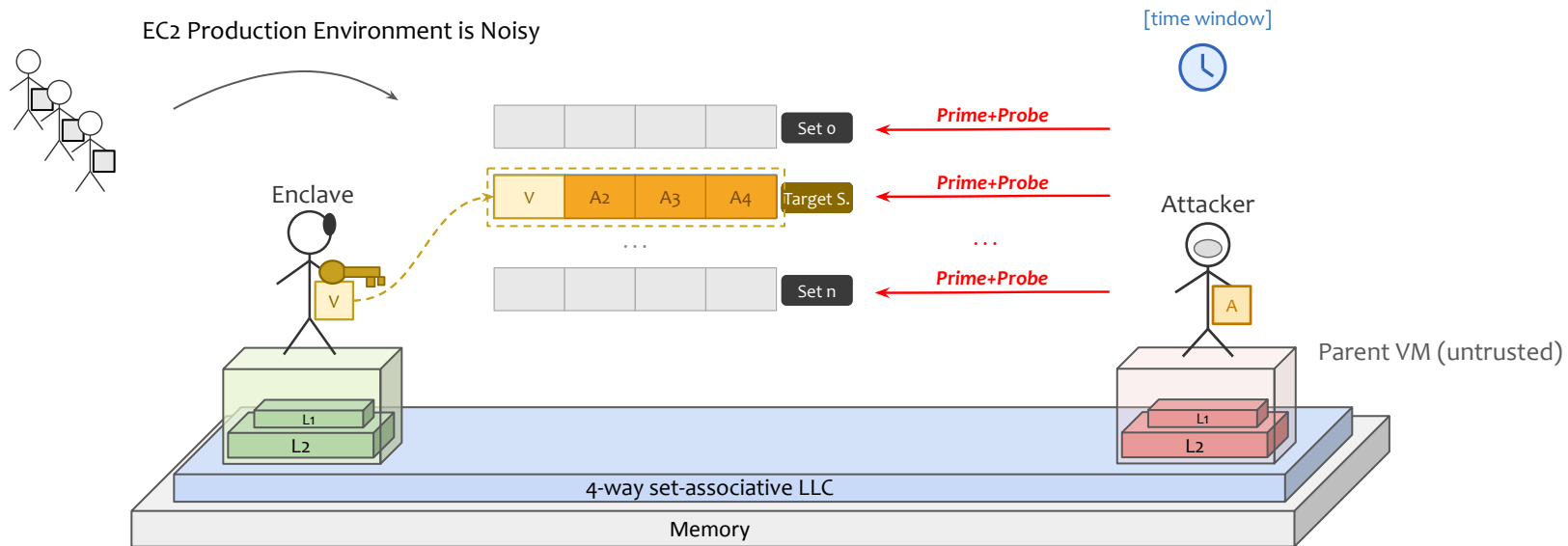
Training-free noise-resilient information extraction



Huawei Qingtian Enclave and Intel SGX affected



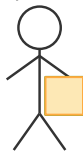
Identifying Target Set



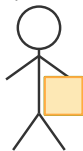
Identifying Target Set

Frequency-domain analysis = noise-tolerant fingerprint

noisy co-tenant



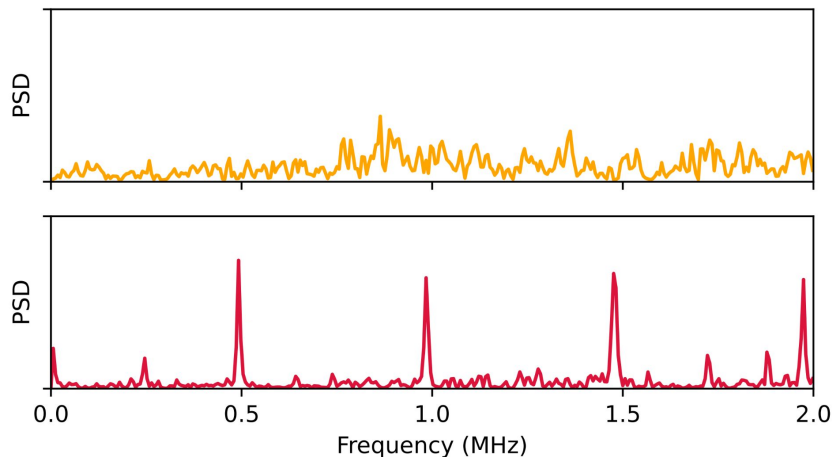
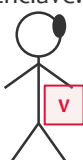
noisy co-tenant



noisy co-tenant



Enclave!



Example Target Trace

no training needed for classification
peak-based heuristic → 71.57% detection rate ≤ 3 scan passes

Does It Actually Work?

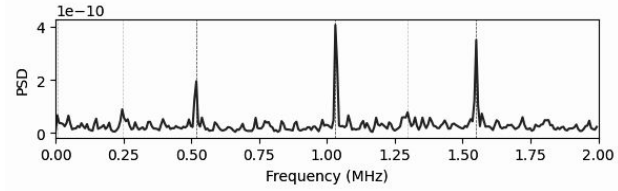
Experimental setup:

- r6i.2xlarge EC2 instance
- Intel Ice Lake-SP, 8 vCPUs
- 1 full core allocated to Nitro Enclave

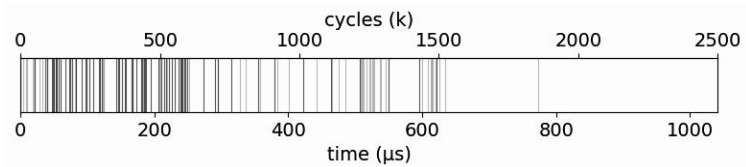
Confidential workloads:

1. Bit-conditional memory accesses over 128 bit secret
2. Vulnerable OpenSSL 1.0.1e ECDSA Montgomery ladder multiplication by 571-bit secret nonce (sect571r1)

PoC

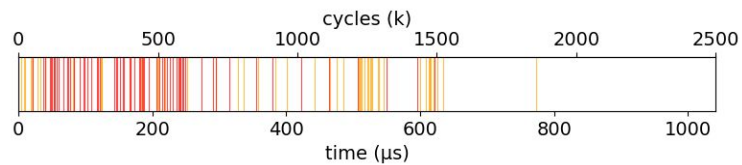


PoC

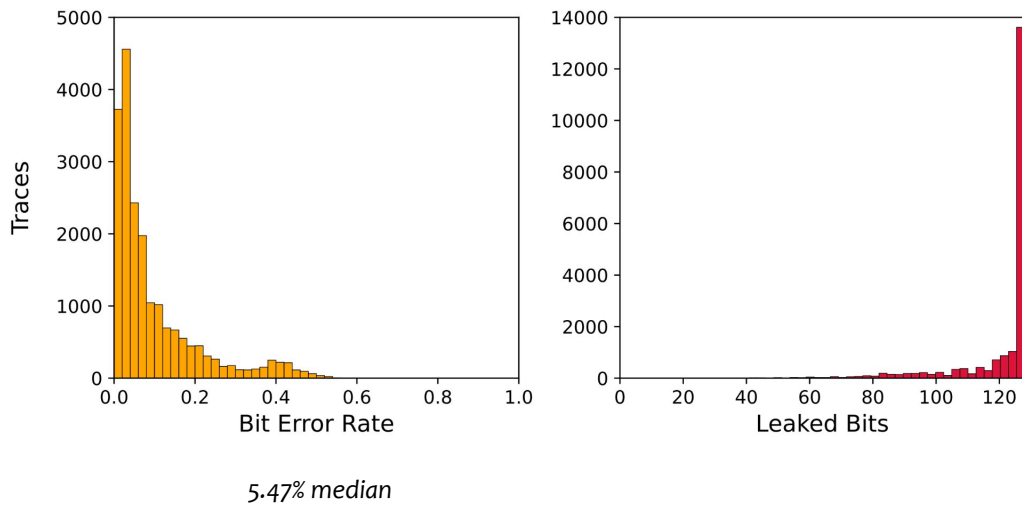


20k target traces

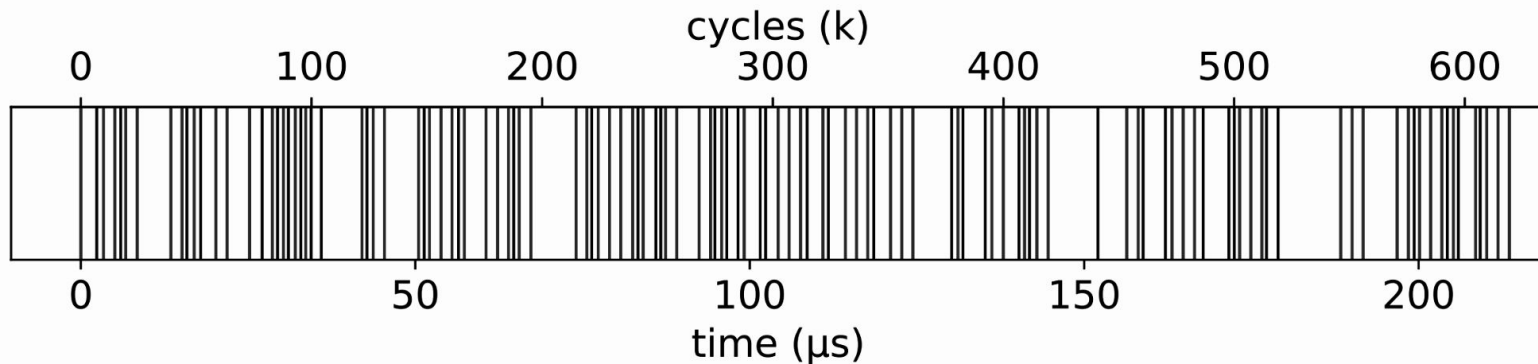
PoC



20k target traces



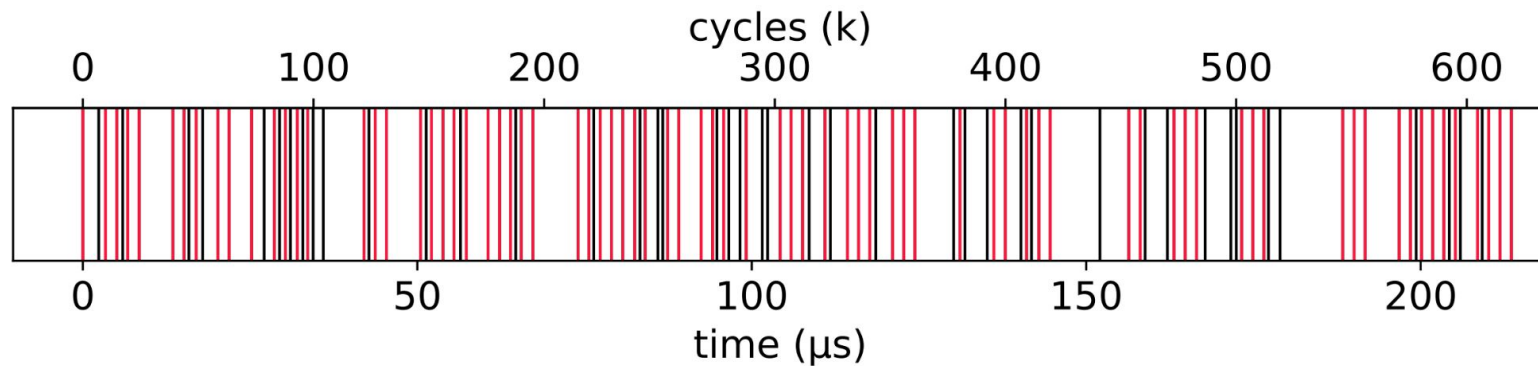
Key Extraction



Prior work: supervised classification (training data, noise shifts, limited interpretability)

1. minimize deviations from nearest periodic positions (L_2 loss)
2. k -means to separate victim accesses from noise
3. interpolate rest

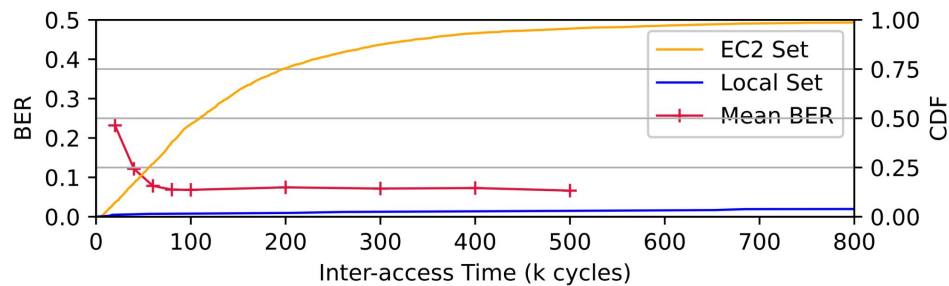
Key Extraction



no training + noise-resilient + interpretable

Output: secret bitstream

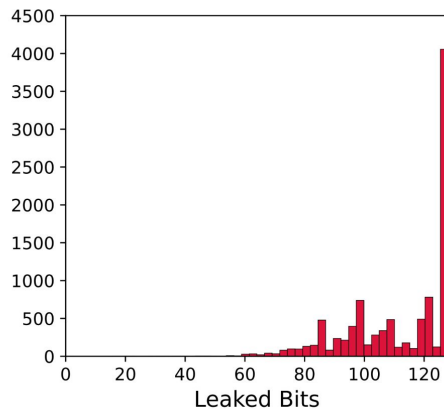
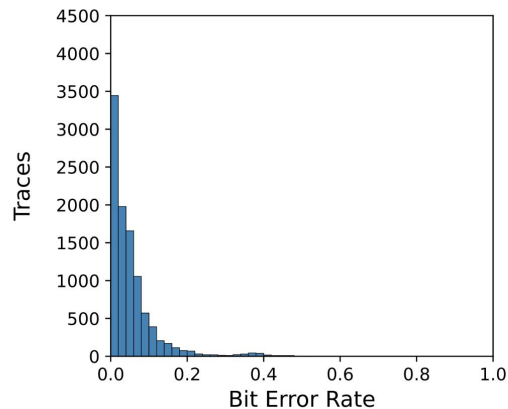
Noise



target detection stays high, extraction degrades but remains practical at EC2-like noise levels

Other enclave technologies

Intel SGX, Huawei Qingtian also **share LLC** with host/co-located VMs → vulnerable



Intel SGX
(local, noise-free)

- AWS & Huawei confirmed the vulnerability
- AWS recommends dedicated hosts as mitigation — at higher cost

PoC: OpenSSL 1.0.1e ECDSA Montgomery ladder multiplication by secret nonce

```
for (; i >= 0; i--)
{
    word = scalar->d[i];
    while (mask)
    {
        if (word & mask)
        {
            if (!gf2m_Madd(group, &point->X, x1, z1, x2, z2, ctx)) goto err;
            if (!gf2m_Mdouble(group, x2, z2, ctx)) goto err;
        }
        else
        {
            if (!gf2m_Madd(group, &point->X, x2, z2, x1, z1, ctx)) goto err;
            if (!gf2m_Mdouble(group, x1, z1, ctx)) goto err;
        }
        mask >>= 1;
    }
    mask = BN_TBIT;
}
```

```

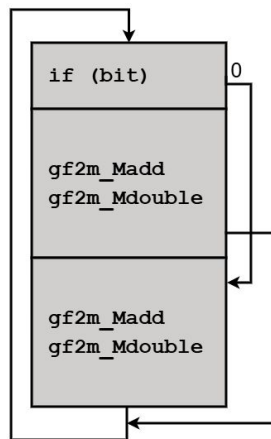
ecdsa/openssl:      file format elf64-x86-64
Disassembly of section .text:
0000000000013a387 <ec_GF2m_montgomery_point_multiply+0x25b>:
CL=14 |0x13a387: | /-----> 48 8b 45 f8      mov     rax,QWORD PTR [rbp-0x8]
      |0x13a38b: |         | 48 23 45 d0      and     rax,QWORD PTR [rbp-0x30]
      |0x13a38f: |         | 48 85 c0       test    rax,rax
      |0x13a392: |         | /-- 74 5e      je      13a3f2
      <ec_GF2m_montgomery_point_multiply+0x2c6>
      |0x13a394: |         | 48 8b 45 a0      mov     rax,QWORD PTR [rbp-0x60]
      |0x13a398: |         | 48 8d 70 08     lea     rsi,[rax+0x8]
      |0x13a39c: |         | 4c 8b 45 f0      mov     r8,QWORD PTR [rbp-0x10]
      |0x13a3a0: |         | 48 8b 7d e8      mov     rdi,QWORD PTR [rbp-0x18]
      |0x13a3a4: |         | 48 8b 4d e0      mov     rcx,QWORD PTR [rbp-0x20]
      |0x13a3a8: |         | 48 8b 55 d8      mov     rdx,QWORD PTR [rbp-0x28]
      |0x13a3ac: |         | 48 8b 45 b8      mov     rax,QWORD PTR [rbp-0x48]
      |0x13a3b0: |         | 48 83 ec 08     sub     rsp,0x8
      |0x13a3b4: |         | ff 75 98      push   QWORD PTR [rbp-0x68]
      |0x13a3b7: |         | 4d 89 c1      mov     r9,r8
      |0x13a3ba: |         | 49 89 f8      mov     r8,rdi
      |0x13a3bd: |         | 48 89 c7      mov     rdi,rax
CL=15 |0x13a3c0: |         | e8 0b f7 ff ff      call   139ad0 <gf2m_Madd>
      |0x13a3c5: |         | 48 83 c4 10     add     rsp,0x10
      |0x13a3c9: |         | 85 c0       test    eax,eax
      |0x13a3cb: |         | 0f 84 7c 01 00 00     je      13a54d
      <ec_GF2m_montgomery_point_multiply+0x421>
      |0x13a3d1: |         | 48 8b 4d 98      mov     rcx,QWORD PTR [rbp-0x68]
      |0x13a3d5: |         | 48 8b 55 f0      mov     rdx,QWORD PTR [rbp-0x10]
      |0x13a3d9: |         | 48 8b 75 e8      mov     rsi,QWORD PTR [rbp-0x18]
      |0x13a3dd: |         | 48 8b 45 b8      mov     rax,QWORD PTR [rbp-0x48]
      |0x13a3e1: |         | 48 89 c7      mov     rdi,rax
      |0x13a3e4: |         | e8 37 f5 ff ff      call   139920 <gf2m_Mdouble>
      |0x13a3e9: |         | 85 c0       test    eax,eax
      |0x13a3eb: |         | /--|-- 75 62      jne     13a44f
      <ec_GF2m_montgomery_point_multiply+0x323>
      |0x13a3ed: |         | e9 68 01 00 00     jmp     13a55a
      <ec_GF2m_montgomery_point_multiply+0x42e>
      |0x13a3f2: |         | \-> 48 8b 45 a0      mov     rax,QWORD PTR [rbp-0x60]
      |0x13a3f6: |         | 48 8d 70 08     lea     rsi,[rax+0x8]
      |0x13a3fa: |         | 4c 8b 45 e0      mov     r8,QWORD PTR [rbp-0x20]
      |0x13a3fe: |         | 48 8b 7d d8      mov     rdi,QWORD PTR [rbp-0x28]
CL=16 |0x13a402: |         | 48 8b 4d f0      mov     rcx,QWORD PTR [rbp-0x10]
      |0x13a406: |         | 48 8b 55 e8      mov     rdx,QWORD PTR [rbp-0x18]
      |0x13a40a: |         | 48 8b 45 b8      mov     rax,QWORD PTR [rbp-0x48]
      |0x13a40e: |         | 48 83 ec 08     sub     rsp,0x8
      |0x13a412: |         | ff 75 98      push   QWORD PTR [rbp-0x68]
      |0x13a415: |         | 4d 89 c1      mov     r9,r8
      |0x13a418: |         | 49 89 f8      mov     r8,rdi
      |0x13a41b: |         | 48 89 c7      mov     rdi,rax
      |0x13a41e: |         | e8 ad f6 ff ff      call   139ad0 <gf2m_Madd>
      |0x13a423: |         | 48 83 c4 10     add     rsp,0x10
      |0x13a427: |         | 85 c0       test    eax,eax
      |0x13a429: |         | 0f 84 21 01 00 00     je      13a550
      <ec_GF2m_montgomery_point_multiply+0x424>
      |0x13a42f: |         | 48 8b 4d 98      mov     rcx,QWORD PTR [rbp-0x68]
      |0x13a433: |         | 48 8b 55 e0      mov     rdx,QWORD PTR [rbp-0x20]
      |0x13a437: |         | 48 8b 75 d8      mov     rsi,QWORD PTR [rbp-0x28]
      |0x13a43b: |         | 48 8b 45 b8      mov     rax,QWORD PTR [rbp-0x48]
      |0x13a43f: |         | 48 89 c7      mov     rdi,rax
CL=17 |0x13a442: |         | e8 d9 f4 ff ff      call   139920 <gf2m_Mdouble>
      |0x13a447: |         | 85 c0       test    eax,eax
      |0x13a449: |         | 0f 84 04 01 00 00     je      13a553
      <ec_GF2m_montgomery_point_multiply+0x427>
      |0x13a44f: |         | \-----> 48 d1 6d d0      shr     QWORD PTR [rbp-0x30],1
      |0x13a453: |         | 48 83 7d d0 00     cmp     QWORD PTR [rbp-0x30],0x0
      |0x13a458: |         | 9e 95 00 f6 f6 ff      cmc     r8

```

14

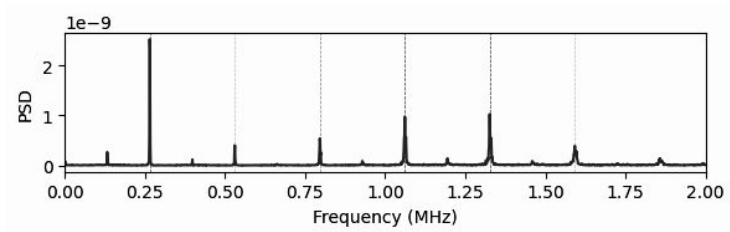
15

16

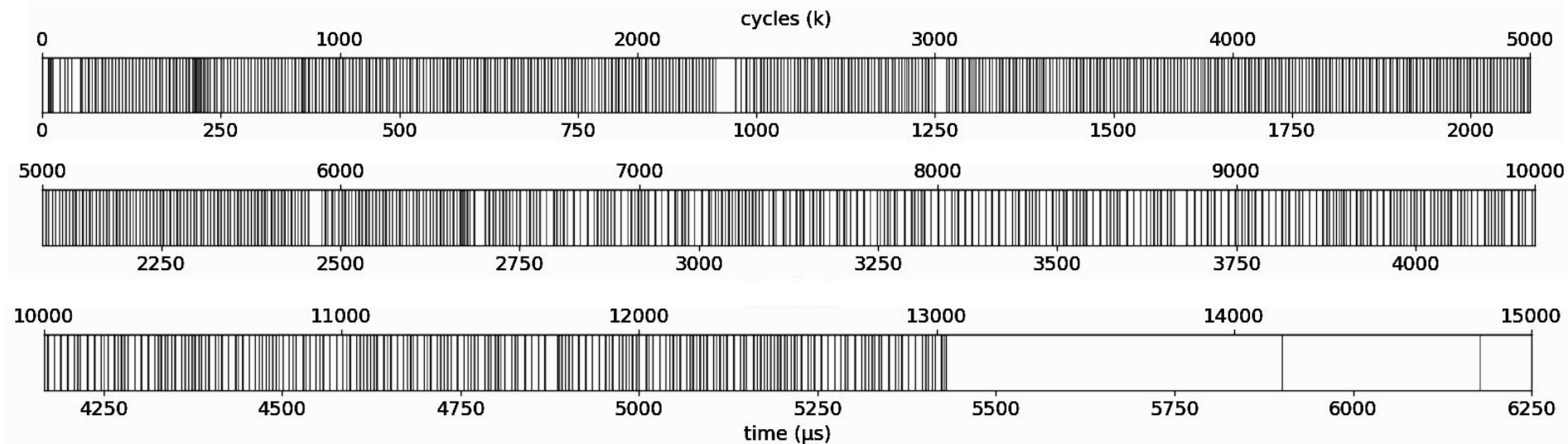


We attack
cache line 15

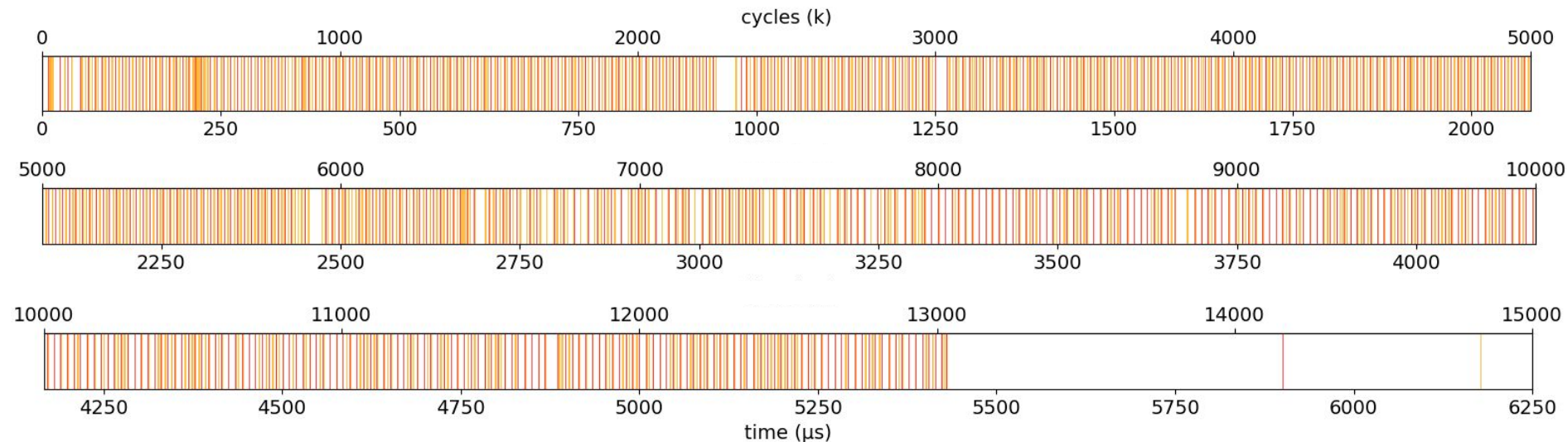
PoC: Vulnerable OpenSSL 1.0.1e ECDSA Montgomery ladder multiplication by secret nonce



PoC: Vulnerable OpenSSL 1.0.1e ECDSA Montgomery ladder multiplication by secret nonce



PoC: Vulnerable OpenSSL 1.0.1e ECDSA Montgomery ladder multiplication by secret nonce



[*] Recovery Results:

Nonce: 0x07ffcc39c31feab30e6100b3630d0470a3d8fbb39422c3bd27aa2e9acdd0705d3765fd09c8d
Leak: 0x6fffffffffffffffffffffe5fffffeffffffffffffffffffffffffffff7ffff7ffe3de31f6100b3630d0470a3d8fbb39422c3bd27aa2e9acdd0305d3765fd09c8d
Leak Len: 577 bits
BER: 0.0333 (3.33%)
Accuracy: 96.67%
Bit Errors: 19 / 577

Mitigations

Challenges: Scalability, energy overhead, coverage against adaptive adversaries

- Dedicated NUMA node: Enclave on one node, parent EC2 on another → strong isolation but resource-intensive
- Hardware cache partitioning: strict LLC separation between enclave VM and all other VMs

Mitigations

Category	Mechanism	Description	Pros	Cons	Overhead	Applicable
HW	Cache Partitioning	Divides LLC into isolated segments (e.g., Intel CAT)	Strong isolation	Reduces efficiency; requires hardware support	Low-moderate (~4%)	Cache/LLC
HW	Cache Randomization	Randomizes mappings/replacement (ScatterCache, CEASER)	Masks patterns	Incomplete vs flush attacks	Low (5-10%)	Both
HW	Replacement Policies	Alters eviction (Random-LRU, SHRP)	Security without redesign	Possible performance impact	Low-moderate	LLC focus
HW	Pseudo-Locking	Pins sensitive data (CATalyst)	Constant-time access	Limited scope; setup cost	<1-17%	LLC
HW	Non-Inclusive Cache	LLC doesn't evict private caches	Mitigates some attacks	Not complete	Negligible	LLC
SW	Constant-Time Code	Uniform execution/memory access (e.g., branchless crypto)	No hardware changes	Requires code rewrites	Variable	Both
SW	Randomization	Random delays/dummy accesses	Obscures timing	Performance hit in noisy environments	5-35%	Both
SW	Cache Flushing	Clears cache lines after use	Removes residual data	Slows execution; not preventive	Moderate	Both
SW	Preloading/Pinning	Preloads data; uses HTM for pinning	Ensures cache hits	Limited scalability	Low	Both
SW	Speculative Controls	Barriers (lfence), SLH, data relocation	Stops Spectre-like attacks	High performance cost	Up to 2x	Both
HW+SW	Scheduling/Isolation	Avoids co-location; disables SMT; core affinity	Broad protection	Reduces parallelism	Low-moderate	Both
HW+SW	Noise Injection	Random access threads (Ghost Thread)	Harder attacks	Runtime overhead	<1%	Both
HW+SW	Attack Detection	Uses counters (PCM, TSX)	Enables response; low false positives	Reactive only	Low	Both
HW+SW	Domain Protections	MPK / domain IDs (Ghostbusting, IVcache)	Fine-grained control	Needs OS support	Low-moderate	LLC hybrids
HW+SW	Measurement Randomization	Extends ASLR to cache sets	Disrupts eviction-set building	Limited vs adaptive attackers	Low	Both

Summary

Uncovered a **LLC-sharing vulnerability** between AWS Nitro Enclaves and parent EC2 VMs

Demonstrated **end-to-end cache side-channel attacks** on real enclave workloads

Extended attack to other enclave technologies — both AWS & Huawei confirmed the vulnerability

Try it out!

<https://codeberg.org/monder/LLCEnclave>